MACHINE LEARNING PRACTICAL NOTES

Answer-sheet-ready notes for all five assignments

Suggested answer-sheet format: Aim → Notes/Theory → Important formula → Algorithm steps (if asked) → Output → Conclusion.

1. Uber Fare Prediction – Linear Regression and Random Forest Regression

Aim: To predict the price of an Uber ride from a given pickup point to the agreed drop-off location.

Notes / Theory

1. Uber fare prediction is a **supervised machine learning** problem. The model learns from labelled examples to predict a fare.

2. Since fare is a continuous numerical value, this is a **regression** problem.

3. Data preprocessing prepares the dataset by handling missing values, examining outliers and selecting useful features.

4. The **Haversine formula** calculates the great-circle distance between pickup and drop-off coordinates using latitude and longitude.

5. Correlation measures the strength and direction of the relationship between two variables. Its value ranges from -1 to +1.

6. **Linear Regression** models a linear relationship between input features and the fare.

7. **Random Forest Regression** combines predictions from multiple decision trees and averages them to produce a final prediction.

8. The dataset is divided into training and testing sets. Training data fits the model; testing data evaluates it on unseen records.

9. MAE, MSE, RMSE and R^2 score are common regression evaluation metrics. Lower error and a higher R^2 generally indicate a better fit.

Important formulas

- Linear Regression: $\mathbf{y = mx + c}$
- Standardisation: $\mathbf{z = (x - \mu) / \sigma}$
- Haversine: $\mathbf{a = \sin^2(\Delta\phi/2) + \cos(\phi■)\cos(\phi■)\sin^2(\Delta\lambda/2)}$; $\mathbf{d = 2R \arcsin(\sqrt{a})}$

Time complexity

Training complexity depends on the implementation, number of features and model settings; Random Forest cost also depends on the number and depth of trees.

Conclusion

Uber fare can be predicted using regression algorithms. Linear Regression and Random Forest Regression can be compared using regression metrics.

2. Email Spam Classification – KNN and SVM

Aim: To classify email messages as Spam or Not Spam using K-Nearest Neighbours and Support Vector Machine.

Notes / Theory

1. Email spam detection is a **supervised classification** problem because the training data contains labelled examples.

2. The target has two categories, Spam and Not Spam, so it is a **binary classification** problem.

3. Preprocessing converts email information into a numerical representation suitable for machine learning.

4. **K-Nearest Neighbours (KNN)** classifies a new email using the majority class among its K nearest training samples.

5. **Support Vector Machine (SVM)** finds a decision boundary called a hyperplane to separate different classes.

6. Support vectors are the data points closest to the hyperplane. They help determine the position of the decision boundary.

7. The data is split into training and testing sets. Models learn from training data and are evaluated on testing data.

8. A confusion matrix compares actual and predicted classes using True Positive, True Negative, False Positive and False Negative.

9. Accuracy, precision, recall and F1-score evaluate classification. Precision matters when avoiding genuine emails being marked as spam; recall matters when catching more spam.

Important formulas

- Euclidean distance: **d = √[Σ(x■ – y■)²]**
- Accuracy: **(TP + TN) / (TP + TN + FP + FN)**
- Precision: **TP / (TP + FP)**; Recall: **TP / (TP + FN)**
- F1-score: **2 × (Precision × Recall) / (Precision + Recall)**

Time complexity

KNN prediction is commonly O(n × d) per query when distances to all n training samples across d features are calculated. SVM complexity depends on the kernel, data and solver.

Conclusion

KNN and SVM classify emails into Spam and Not Spam categories. Their performance is compared using classification metrics.

3. Bank Customer Churn Prediction – Neural Network

Aim: To predict whether a bank customer is likely to leave the bank within the next six months using customer information and a neural network.

Notes / Theory

1. **Customer churn** occurs when a customer stops using a company's or bank's services.

2. Churn prediction is supervised binary classification when historical records contain a churn label.

3. Preprocessing prepares data for learning. Categorical features may require encoding and numerical features may require scaling.

4. The dataset is divided into training and testing sets. Training data is used to learn; testing data evaluates performance.

5. An **Artificial Neural Network (ANN)** is a model made of connected artificial neurons arranged in layers.

6. The **input layer** receives features, **hidden layers** learn patterns, and the **output layer** produces the prediction.

7. A **Dense layer** is fully connected: each neuron receives input from every neuron in the previous layer.

8. **ReLU** is commonly used in hidden layers. It returns zero for negative inputs and the input itself for positive inputs.

9. **Sigmoid** maps a value between 0 and 1 and is commonly used for a binary-classification output.

10. **Binary Crossentropy** is a loss function for binary classification. The **Adam optimizer** updates weights to reduce loss.

11. An **epoch** is one complete pass through the training dataset. Batch size is the number of samples processed before a weight update.

12. Accuracy and the confusion matrix can evaluate the model. Overfitting occurs when training performance is good but unseen-data performance is poor.

Important formulas

- ReLU: **$f(x) = \max(0, x)$**
- Sigmoid: **$f(x) = 1 / (1 + e^{-x})$**

Time complexity

Training cost depends on the number of samples, input features, neurons, layers, epochs and batch size.

Conclusion

A neural network can learn patterns in customer data and classify whether a customer is likely to churn.

4. Diabetes Classification – K-Nearest Neighbours

Aim: To implement KNN on the diabetes dataset and calculate confusion matrix, accuracy, error rate, precision and recall.

Notes / Theory

1. Diabetes classification is a **supervised binary classification** problem that predicts diabetic or non-diabetic class.

2. **K-Nearest Neighbours (KNN)** is a distance-based algorithm. It assigns a class based on the majority class among the K nearest training samples.

3. K represents the number of neighbours considered when making a prediction.

4. The dataset is divided into training and testing sets. KNN uses labelled training samples and is evaluated on testing samples.

5. Feature scaling is important because KNN uses distance calculations. Features with larger numerical ranges could otherwise dominate the distance.

6. Euclidean distance is a commonly used measure of distance between two data points.

7. A confusion matrix compares actual and predicted classes and provides TP, TN, FP and FN values.

8. Accuracy measures the proportion of correct predictions. Error rate measures the proportion of incorrect predictions.

9. Precision measures how many predicted positive cases are actually positive. Recall measures how many actual positive cases are correctly identified.

10. In health screening, recall is important because a false negative means an actual positive case was missed. A model prediction is not a medical diagnosis.

Important formulas

- Euclidean distance: **d = √[Σ(x■ – y■)²]**
- Accuracy: **(TP + TN) / (TP + TN + FP + FN)**
- Error rate: **1** – **Accuracy**
- Precision: **TP / (TP + FP)**; Recall: **TP / (TP + FN)**

Time complexity

A common brute-force KNN prediction costs O(n × d) per query, where n is the number of training samples and d is the number of features.

Conclusion

KNN classifies diabetes records based on similarity between feature values. Performance is measured using the confusion matrix and classification metrics.

5. Customer Segmentation – K-Means Clustering

Aim: To implement K-Means clustering on sales_data_sample.csv and determine the number of clusters using the Elbow Method.

Notes / Theory

1. Customer segmentation groups customers with similar characteristics and can reveal similar purchasing or sales patterns.

2. **K-Means** is an unsupervised learning algorithm that divides data into K clusters based on distance from cluster centroids.

3. Unsupervised learning finds patterns in data without using a target label for each sample.

4. K is the number of clusters. A **centroid** is the mean position of the data points assigned to a cluster.

5. Preprocessing may include removing irrelevant columns, handling missing values and converting categorical data into numerical form.

6. Label Encoding assigns integer labels to categories. StandardScaler standardises numerical features so large-scale features do not dominate distance calculations.

7. K centroids are initialised. Each point is assigned to its nearest centroid, then centroids are recalculated as the mean of assigned points.

8. Assignment and centroid-update steps repeat until clusters or centroids stop changing significantly.

9. **WCSS (Within-Cluster Sum of Squares)** measures the sum of squared distances between each data point and its assigned centroid.

10. The **Elbow Method** runs K-Means for different K values and plots WCSS against K. The elbow is where the reduction in WCSS begins to slow.

Important formulas

- WCSS: $\Sigma\blacksquare\blacksquare\blacksquare\blacksquare\ \Sigma\blacksquare\in C\blacksquare\ ||x - \mu\blacksquare||^2$
- Typical K-Means cost per run: $O(n \times K \times i \times d)$

Time complexity

$O(n \times K \times i \times d)$, where n is the number of data points, K is clusters, i is iterations and d is features.

Conclusion

K-Means groups similar data points into clusters. The Elbow Method helps estimate a suitable number of clusters using WCSS.